```
import pandas as pd
```

```
orders = pd.read_csv("olist_orders_dataset.csv")
order_items = pd.read_csv("olist_order_items_dataset.csv")
order_reviews = pd.read_csv("olist_order_reviews_dataset.csv")
products = pd.read_csv("olist_products_dataset.csv")
customers = pd.read_csv("olist_customers_dataset.csv")
sellers = pd.read_csv("olist_sellers_dataset.csv")
geo = pd.read_csv("olist_geolocation_dataset.csv")
payments = pd.read_csv("olist_order_payments_dataset.csv")
```

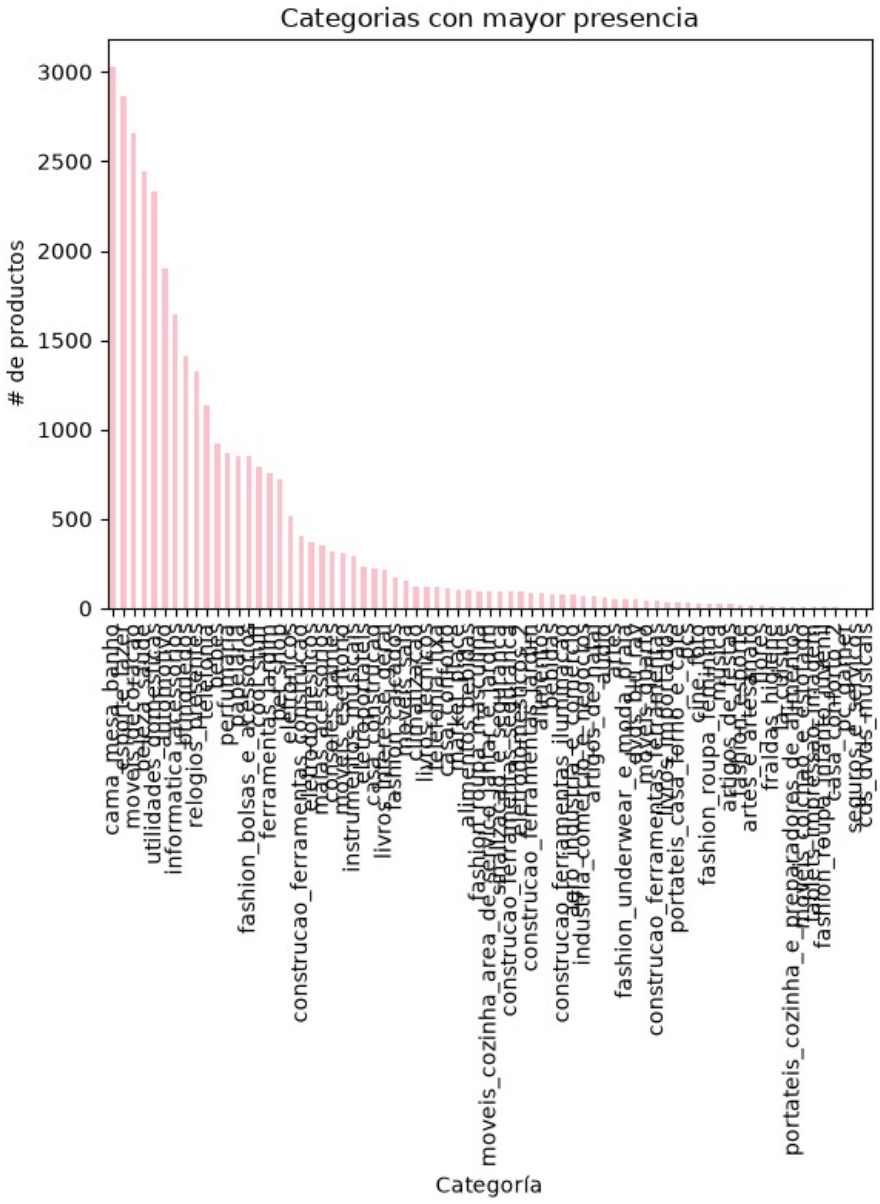
```
orders["order_purchase_timestamp"] = pd.to_datetime(
    orders["order_purchase_timestamp"],
    format="%Y-%m-%d %H:%M:%S",
    errors="coerce"
)

orders["order_delivered_customer_date"] = pd.to_datetime(
    orders["order_delivered_customer_date"],
    format="%Y-%m-%d %H:%M:%S",
    errors="coerce"
)

orders["order_estimated_delivery_date"] = pd.to_datetime(
    orders["order_estimated_delivery_date"],
    format="%Y-%m-%d %H:%M:%S",
    errors="coerce"
)
```

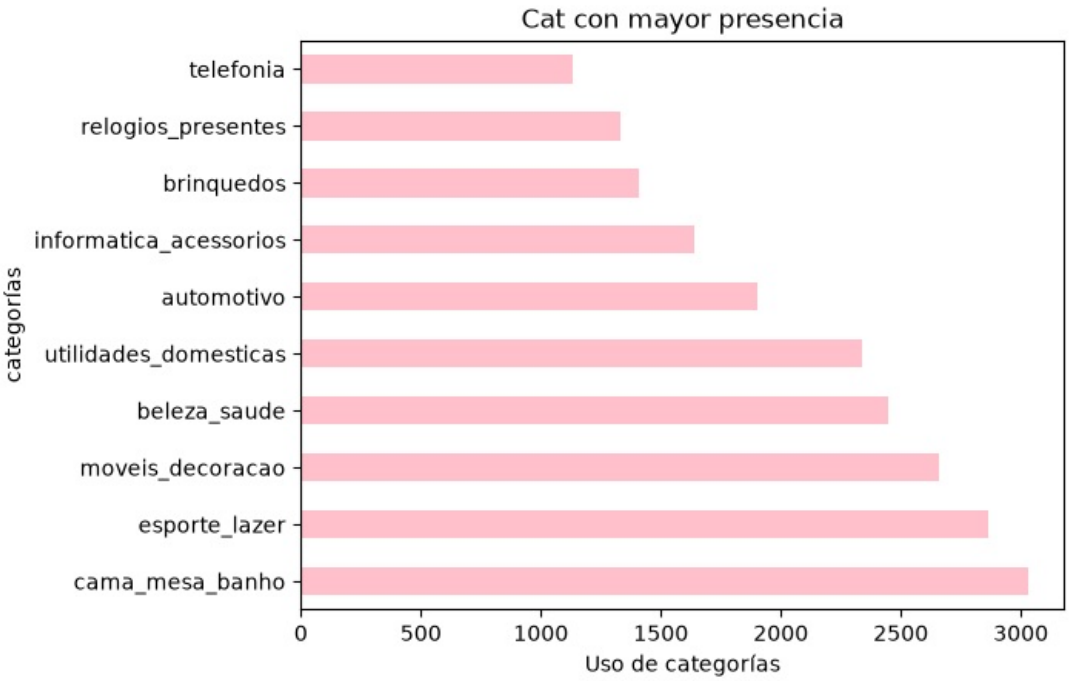
¿Qué categorías tienen mayor presencia en el catálogo de Olist?

```
#Categorias de productos
numero_productos = products["product_category_name"].value_counts()
ax1 = numero_productos.plot(kind="bar", title = "Categorias con mayor presencia", xlabel = "Categoría", ylabel = "# de productos", color = "pink")
```



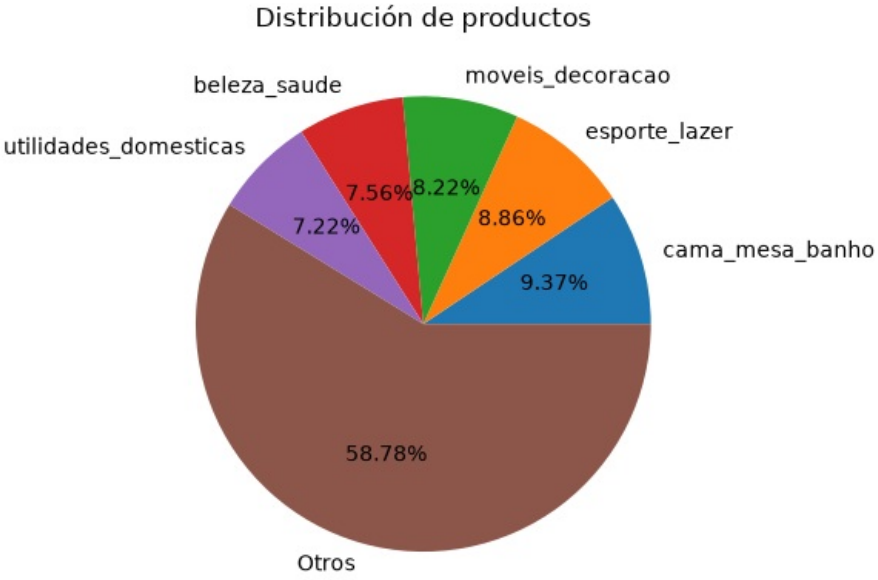
¿Cuáles son las categorías que tienen mayor presencia en el catálogo?

```
#2 Top 10 categorias principales
ax1 = numero_productos.head(10).plot(kind="barh", title = "Cat con mayor presencia", xlabel = "Uso de categorías", ylabel = "categorías", color="pink")
```



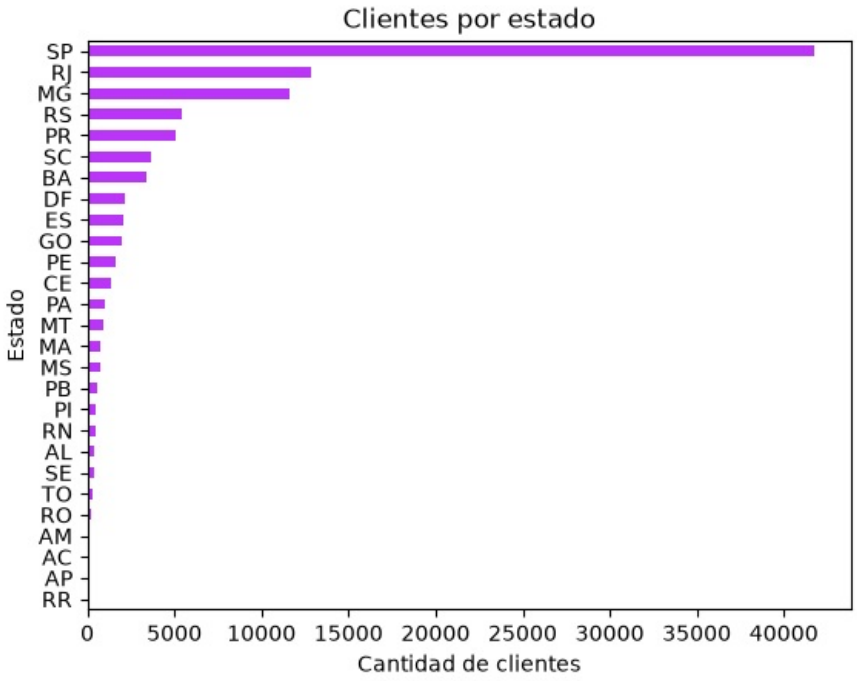
¿Qué proporción del catálogo está concentrada en las principales categorías?

```
#Distribucion de productos
numero_productos = products["product_category_name"].value_counts().head(5)
numero_productos["Otros"] = products["product_category_name"].value_counts().iloc[5:].sum()
ax2 = numero_productos.plot(kind = "pie", autopct="%1.2f%%", title = "Distribución de productos")
```



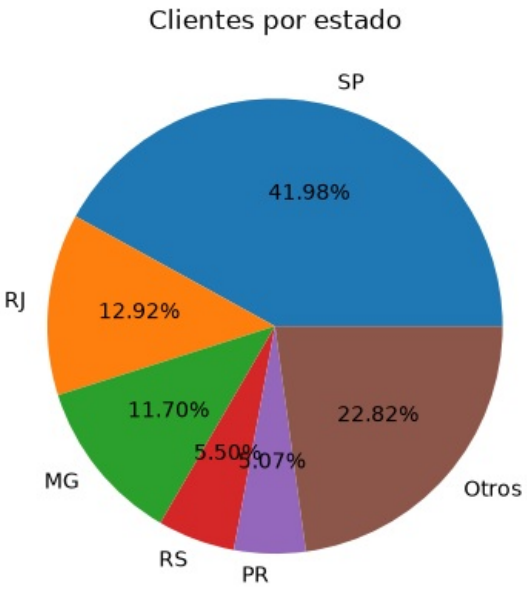
¿Qué estados concentran la mayor cantidad de clientes?

```
#Clientes por estado
cantidad_clientes = customers["customer_state"].value_counts().sort_values(ascending= True)
ax3 = cantidad_clientes.plot(kind="barh", title = "Clientes por estado", color = "#ba35f7", xlabel="Cantidad de clientes", ylabel= "Estado")
```



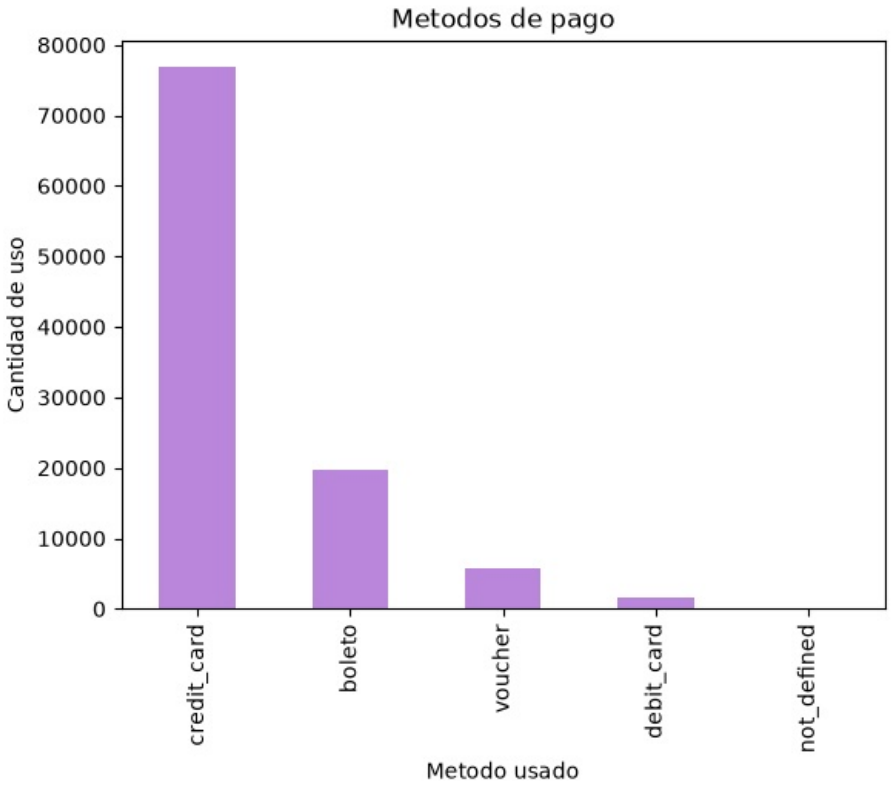

```
### ¿Qué tan concentrada está la base de clientes geográficamente?
```

```
#Estados Principales
cantidad_clientes2 = customers["customer_state"].value_counts()
top5 = cantidad_clientes2.head(5)
top5["Otros"] = cantidad_clientes2.iloc[5:].sum()
ax5 = top5.plot(kind="pie", autopct = "%1.2f%%", title = "Clientes por estado")
```

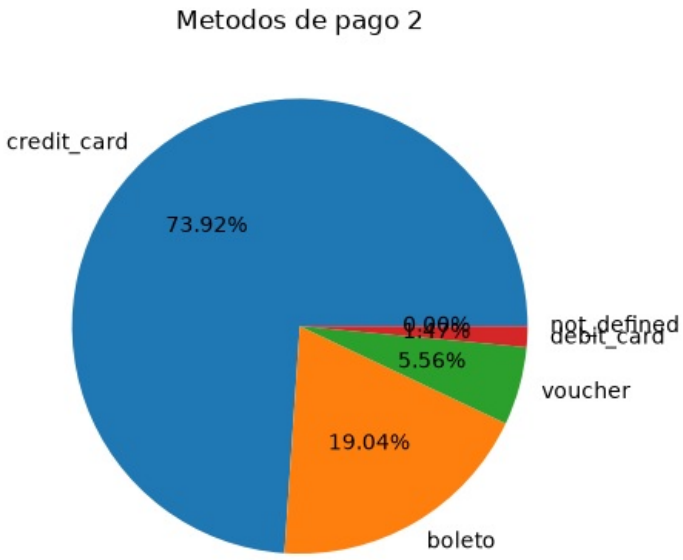


¿Cuál es el método de pago más utilizado?

```
#Metodos de Pago
metodo_de_pago = payments["payment_type"].value_counts()
ax6 = metodo_de_pago.plot(kind="bar", title= "Metodos de pago", color = "#a562d2c5", ylabel = "Cantidad de uso", xlabel = "Metodo usado")
```




```
#Fue la tarjeta de credito
ax6_1 = metodo_de_pago.plot(kind = "pie", title = "Metodos de pago 2", autopct = "%1.2f%%")
```



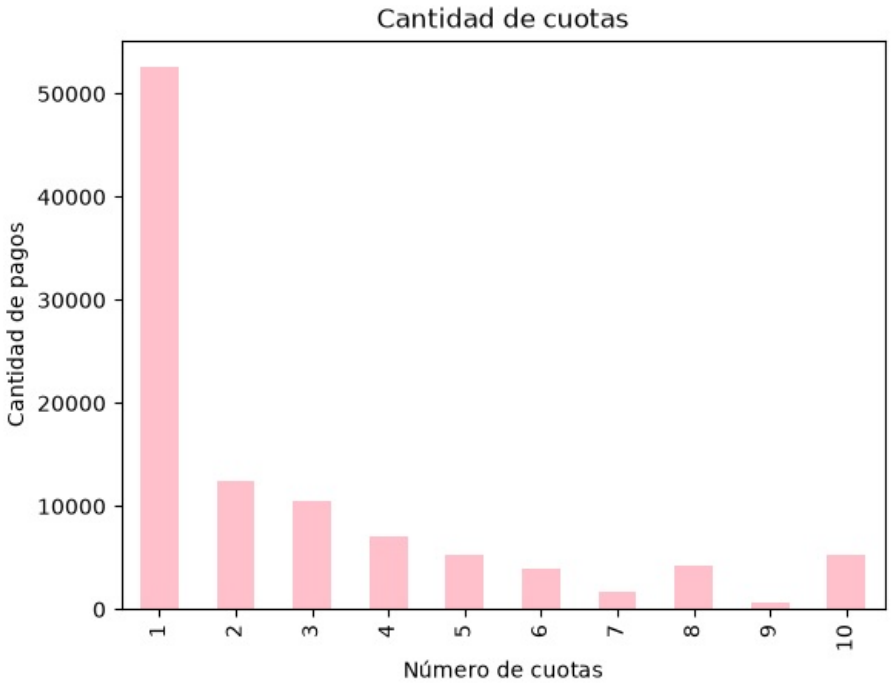
Reflexión:

¿Cuál de las dos gráficas permite comparar mejor los métodos de pago? Para mi opinion el grafico tipo pastel ya que se observa mas claramente que tanto abarca cada metodo de pago

¿Cuál utilizarías si quisieras enfatizar la proporción que representa cada método? Pie

¿Qué número de cuotas es el más frecuente?

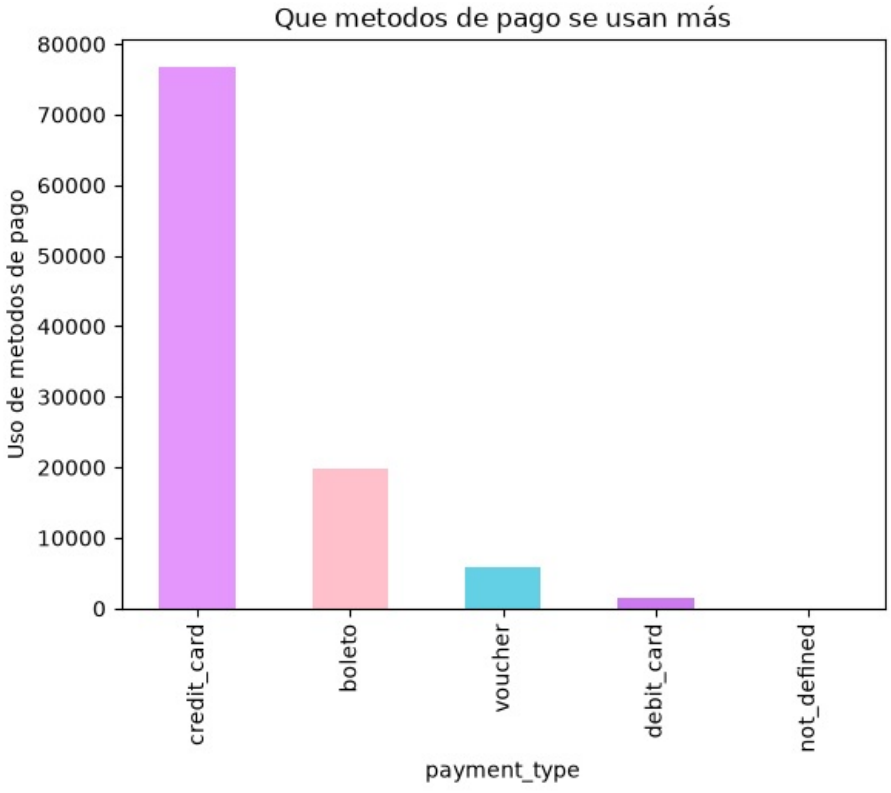
```
#Numero de Cuotas
cantidad_cuotas = payments["payment_installments"].value_counts().sort_index().iloc[1:11]
ax6 = cantidad_cuotas.plot(kind="bar", title = "Cantidad de cuotas", color = "pink", xlabel="Número de cuotas", ylabel= "Cantidad de pagos")
```



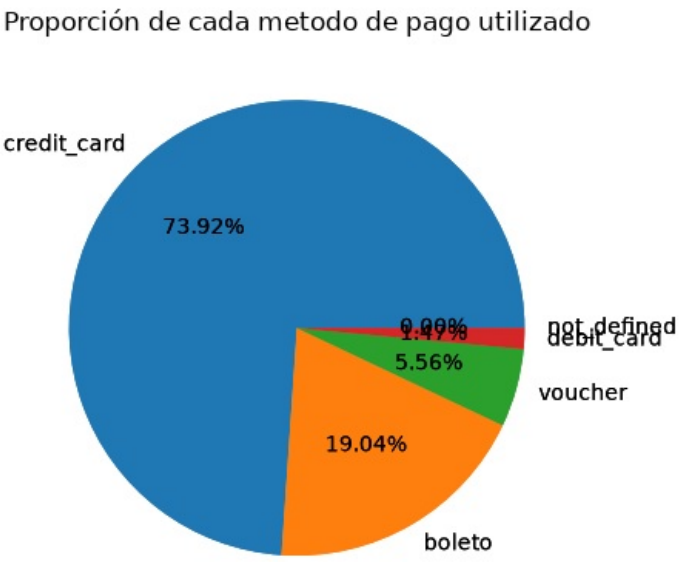
Pregunta: ¿Cuál utilizarías en un reporte para comparar los métodos de pago?

Pregunta: ¿Cuál utilizarías si quisieras enfatizar la proporción que representa cada método?

```
#Comparacion de Graficas
comparacion = payments["payment_type"].value_counts()
ax7 = comparacion.plot(kind= "bar", color =[ "#e596fd","pink","#64d0e5","#cd7af0"])
ax7.set_title("Que metodos de pago se usan más")
ax7.set_xlabel("Metodos de pago")
ax7.set_ylabel("Uso de metodos de pago")
show = comparacion.plot(kind = "bar", color =[ "#e596fd","pink","#64d0e5","#cd7af0"])
```




```
ax7_2 = comparacion.plot(kind = "pie", autopct = "%1.2f%%")
ax7_2.set_title("Proporción de cada metodo de pago utilizado")
ax7_2.set_ylabel("")
show2 = comparacion.plot(kind = "pie", autopct = "%1.2f%%")
```



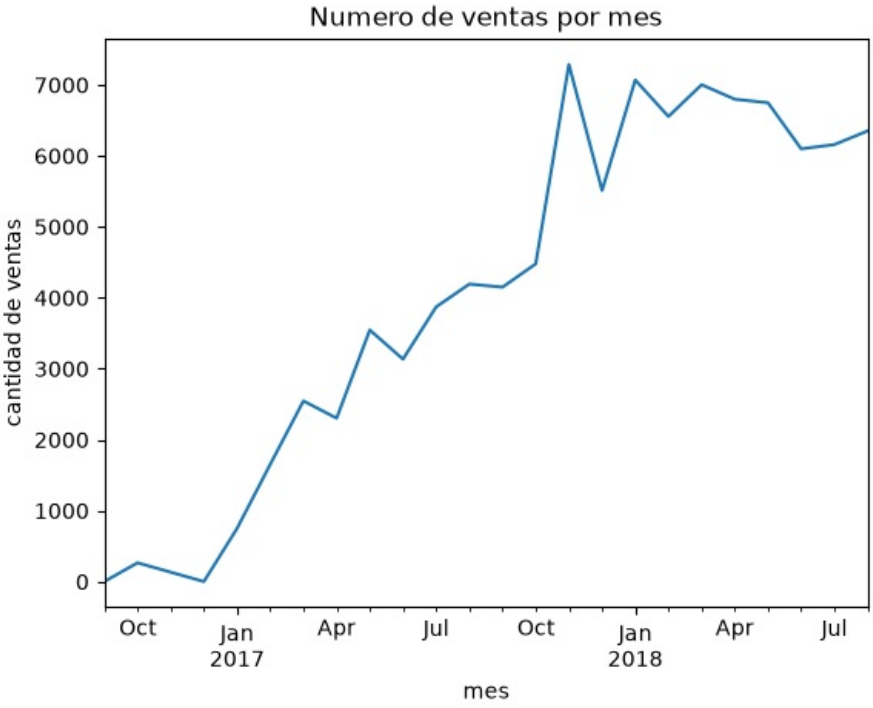
Línea: pedidos por mes

Cargar orders y convertir order_purchase_timestamp a datetime. Filtrar order_status == 'delivered' (opcional). Crear columna de mes con .dt.to_period('M'). groupby por mes y contar order_id con nunique. Recortar meses extremos (2016 y sept–oct 2018 casi no tienen datos): buena lección de "mira tus datos antes de graficar". Graficar mes vs. número de pedidos; título, etiquetas y rotar ticks.

```
#Solo ventas entregadas
df1= orders[orders["order_status"] == "delivered"]
#corto las columnas que necesito
df1= df1[["order_purchase_timestamp","order_id"]]
#Crea una columna mes a partir de "order_purchase_timestamp"
df1["month"] = df1["order_purchase_timestamp"].dt.to_period("M")
#utilizao sample(5) para mostrar 5 elementos aleatorios del dataframe
display(df1.sample(5))
#agrupa por mes y contar order_id
resultado= df1.groupby("month")["order_id"].nunique()
resultado.plot(
kind="line", xlabel="mes", ylabel="cantidad de ventas", title="Numero de ventas por mes"
)
```

	order_purchase_timestamp	order_id	month
7712	2017-10-17 11:54:07	3bce4193e30b36d4d3284df3fa96d77f	2017-10
78319	2018-01-02 12:56:14	6bdd4fc201b1c41b2d0daf3c5a4282ae	2018-01
36845	2017-12-01 16:46:04	2c9692d95147996e657865c7f5ec42fc	2017-12
73178	2018-06-03 18:48:07	09c8e71060b07326dba9db865f46881e	2018-06
54710	2017-11-24 09:53:54	7701820e6e57dbb626a80dfd7ab6f414	2017-11

```
<Axes: title={'center': 'Numero de ventas por mes'}, xlabel='mes', ylabel='cantidad de ventas'>
```



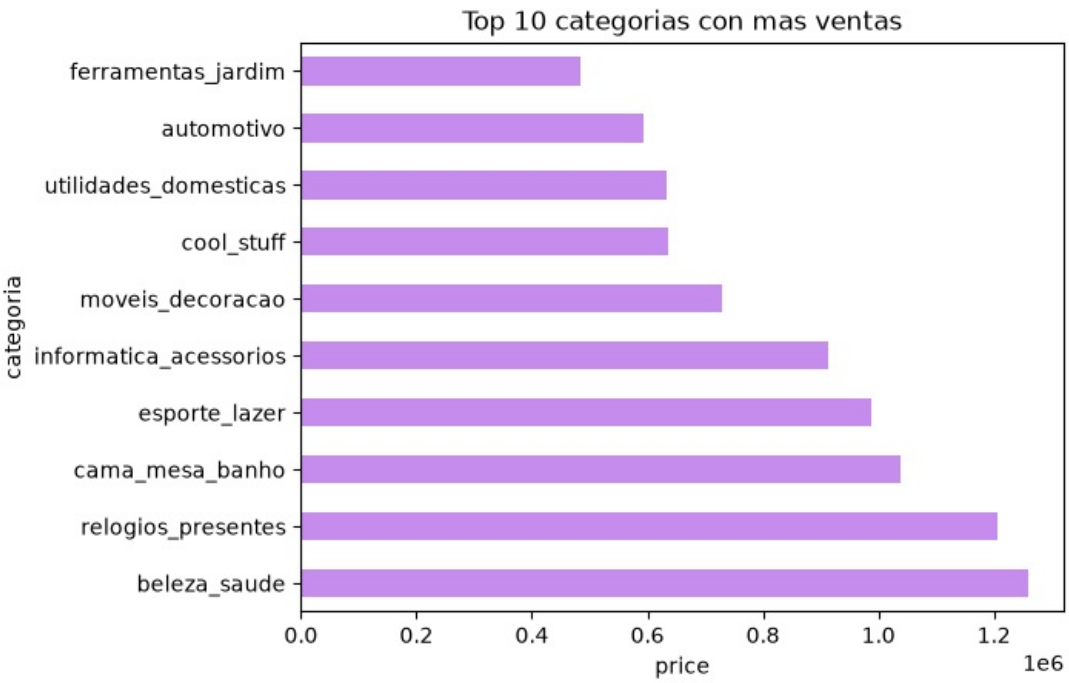
Barras horizontales: top 10 categorías por ingreso

1-.Merge order_items + products por product_id. 2-.groupby categoría → suma de price. 3-.Tomar las 10 mayores con .nlargest(10). Invertir el orden para que la mayor quede arriba y graficar con barh.

```
df_merge = pd.merge(order_items, products, on="product_id")
```

```
df_grouped = df_merge.groupby("product_category_name")["price"].sum()
df_top10 = df_grouped.nlargest(10)
df_top10.plot(kind="barh", color="#c58ced",xlabel="price", ylabel="categoria", title="Top 10 categorias con mas ventas")
```

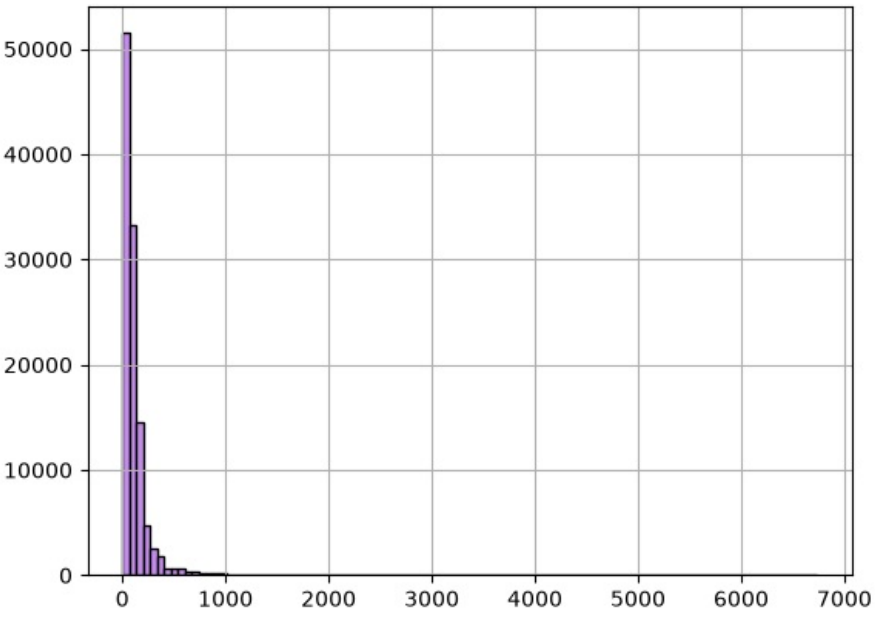
```
<Axes: title={'center': 'Top 10 categorias con mas ventas'}, xlabel='price', ylabel='categoria'>
```



Histograma: distribución del precio

Usar order_items['price']. Primer histograma directo: se verá muy sesgado. Repetir con bins en escala log o con np.log10(price). Discusión: por qué la media engaña. Marcar media y mediana con axvline.

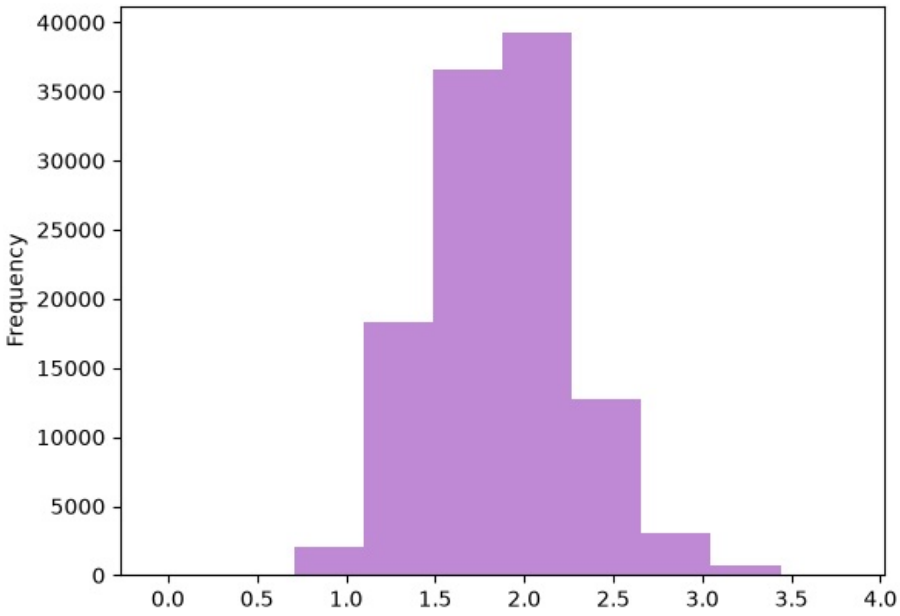
```
# Usar order_items['price'].Primer histograma directo: se verá muy sesgado.
df5= order_items['price'].hist(bins=100, color="#c58ced", edgecolor="black")
```




```
import numpy as np
df5= order_items
df5["log"]= df5["price"].apply(np.log10)
```

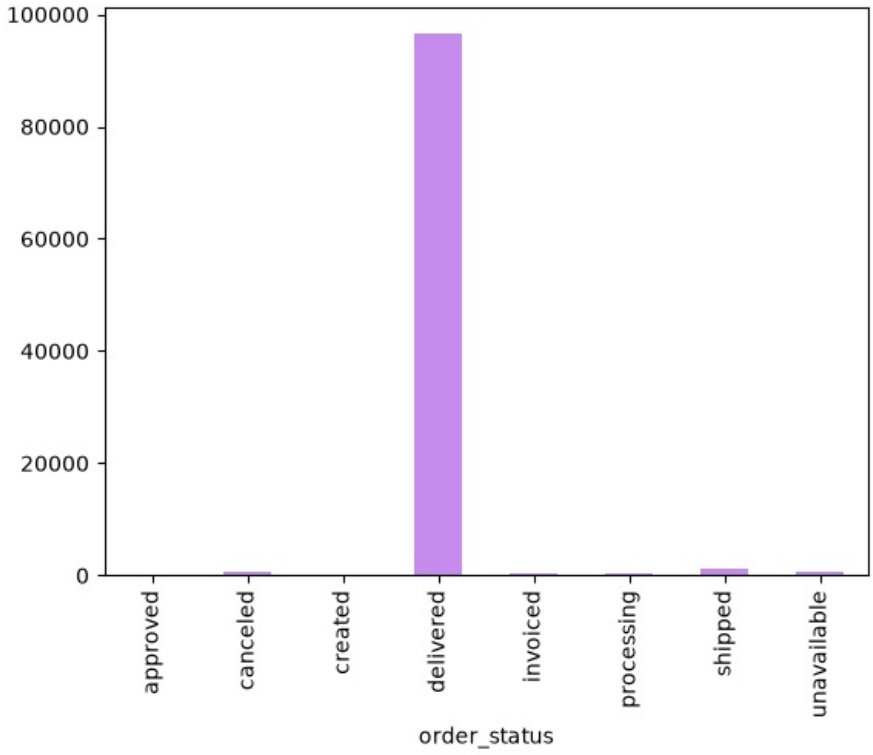
```
df5["log"].plot(kind="hist", color="#bf89d6")
```

```
<Axes: ylabel='Frequency'>
```




```
orders.groupby("order_status")["order_id"].count().plot(kind="bar", color="#c58ced")
```

```
<Axes: xlabel='order_status'>
```



Histograma comparativo: días de entrega reales vs. estimados

1-. En orders, convertir a datetime las columnas de compra, entrega real y entrega estimada. 2-. Calcular dias_reales = entregado − compra y dias_estimados = estimado − compra (en .dt.days). 3-. Eliminar NaN (pedidos no entregados). 4-. (profe) Dos histogramas en el mismo eje con alpha=0.5 y leyenda.

```
#1
orders["order_purchase_timestamp"] = pd.to_datetime(orders["order_purchase_timestamp"])
orders["order_delivered_carrier_date"] = pd.to_datetime(orders["order_delivered_carrier_date"])
orders["order_estimated_delivery_date"] = pd.to_datetime(orders["order_estimated_delivery_date"])

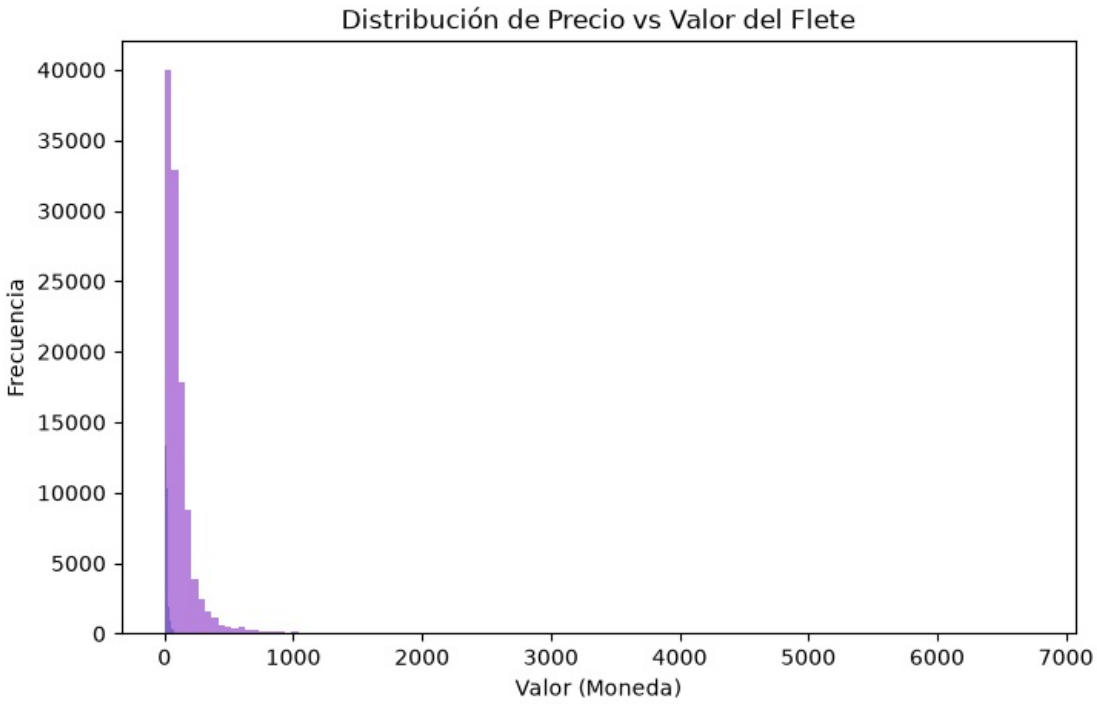
#2
orders["dif_carrier_vs_estimada"] = (orders["order_delivered_carrier_date"] - orders["order_estimated_delivery_date"]).dt.days
orders["dias_estimados"] = (orders["order_estimated_delivery_date"] - orders["order_purchase_timestamp"]).dt.days

#3
orders = orders.dropna(subset=["dif_carrier_vs_estimada", "dias_estimados"])

#4
import matplotlib.pyplot as plt
linzo, pintura= plt.subplots(figsize=(8, 5))
order_items["freight_value"].plot(kind="hist", ax=pintura, alpha=0.6, label="Valor del Flete", bins=130)

order_items["price"].plot(kind="hist",color="#8730c6", ax=pintura, alpha=0.6, label="Precio", bins=130)

pintura.set_title("Distribución de Precio vs Valor del Flete")
pintura.set_xlabel("Valor (Moneda)")
pintura.set_ylabel("Frecuencia")
plt.show()
```



Dispersión: peso del producto vs. costo de envío

1-. Merge order_items + products por product_id. 2-. Tomar product_weight_g y freight_value; quitar NaN. 3-. Muestrear con .sample(3000) (110k puntos saturan la gráfica). 4-. Graficar con alpha bajo; opcionalmente ejes en escala log. Extra: colorear por price con c= y agregar plt.colorbar().

```
order_items = pd.read_csv("olist_order_items_dataset.csv")
products = pd.read_csv("olist_products_dataset.csv")

df = order_items.merge(products, on="product_id", how="left")
df = df.dropna(subset=["product_weight_g", "freight_value"])
muestra = df.sample(n=3000)
muestra.plot(kind="scatter", x="freight_value", y="price", alpha=0.3, c="price", colormap="winter")
```

```
<Axes: xlabel='freight_value', ylabel='price'>
```

